

Counting Versus Testing the Destination: A Simulation Study of Layered Defences Against SMS OTP Flooding and Pumping

Naveed Ahmed^{a,*}

^a *Affiliation to be completed before submission, Saudi Arabia*

Abstract

Endpoints that send one-time passwords by SMS are abused to waste money (flooding) or to earn part of the termination fee (SMS pumping). We evaluate in simulation a twelve-step defence that draws trust from app attestation, signed sessions and verification outcomes, against simpler designs and 25 attacker profiles. Attacks with a signature in the request are contained at little cost to simulated users; a residential attacker with farmed CAPTCHA scores is rationed by refusing half of first-time users. On the destination, short-window counters of SMS sends per number block, selected at a benign service target, leaked 6 to 48 messages against four concentrated 20-minute pumpers at 0.06 to 0.43 points of attack-free completion, where at the busiest density the default sequential tests of verification outcomes leaked 19 to 452. Under a second predeclared protocol on unseen seeds and workloads, the frozen counter met its leakage and benign-cost claims, leaked no more than the tests in all but 13 of 144 cells, all against an instantly verifying carrier, and failed both service-under-attack claims when the pumper shares blocks with real users. For attacks of up to an hour, unfitted estimates ordered the two policies' mean leakage in all 18 decisive boundary cells: the counter loses its advantage against paced or widely spread pumpers.

Keywords: SMS pumping, artificially inflated traffic, one-time password, rate limiting, sequential tests, fraud prevention

*Corresponding author.

Email address: optimistbs@gmail.com (Naveed Ahmed)

1. Introduction

Most online services verify a phone number by sending it a code by SMS, from an endpoint reachable before any account exists, at a tenth of a cent to twenty cents a message. A flooder makes the service send to numbers it does not own; a pumper sends to a range whose operator shares the termination fee, turning the service’s SMS budget into its income [1]. Twitter’s owner said in 2023 that pumping had cost the company about 60 million dollars a year [2], and Huh et al. found as many attack requests as genuine ones in three months of one large service’s logs [3].

This paper starts from such an incident on a registration flow the author operated, and from the redesign that followed: a twelve-step pipeline with attested platforms, signed sessions, number checks before a message is paid for, reputation from whether codes were entered, a five-response risk score and a hard budget. We ask in simulation which attacks it contains against simpler designs, at what cost in completed registrations (RQ1); which layer is responsible (RQ2); when outcome-based reputation separates an attacker from simulated users (RQ3); what rationing costs when it does not (RQ4); and what destination policies cost legitimate users, with and without an attack (RQ5).

The destination became the centre of the study, because a pumper is paid only on its partner’s ranges: under a predeclared comparison a plain counter of sends per number block leaked less than our sequential tests of code outcomes, and a second predeclared protocol mapped where that holds and fails. We contribute eight gaps identified through recollection and subsequent design review, as motivation only, the design and an open implementation, measurements of where reputation stops separating and what rationing depends on, both destination-policy comparisons with their failures and collateral damage, and unfitted estimates of when counting leaks less than our tests; Sections 7.1–7.4 are context for the destination results (Sections 7.5–7.7).

2. Background and related work

Revenue-share fraud. In international revenue share fraud, traffic is generated to an advertised number range and the termination fee is split along the route [4]. Sahin and Francillon’s 3.14 million test numbers from twelve providers show the range to be the fraudster’s managed asset [5].

36 *Detecting artificially inflated SMS.* Huh et al. trained gradient-boosted trees
 37 on 9.4 million SMS authentication logs from nineteen countries, reaching
 38 89.6% recall at 0.19% false positives on web traffic [3]. About 73% of web
 39 attacks sent four or fewer requests per identity a day, defeating per-entity
 40 limits, and destination signals were the strongest: about 92% zero conversion
 41 for attacks against 21% for genuine traffic. Their detector needs labelled
 42 traffic and acts per request on request, history and country features. Their
 43 data show that destination aggregation is informative; what we add is a
 44 controlled comparison of two label-free ways to act on it per block, from
 45 send counts or code outcomes, and of their cost to users who share a block.
 46 Theirs was not reproduced, so the two are not ranked.

47 *Defences in practice.* Twilio’s Fraud Guard blocks regions known for pump-
 48 ing [6], and its SMS Pumping Risk Score advises friction from 60 and refusal
 49 above 90 [7], so graded destination action is established practice; no product
 50 was benchmarked. CAPTCHA solving is a labour market at about a dollar
 51 per thousand [8], open where machine solvers fail [9]; client puzzles [10] were
 52 not evaluated. Residential proxy addresses rotate within minutes [11], and
 53 carrier-grade NAT makes address reputation on cellular networks “all but
 54 infeasible” [12].

55 *Identity and change detection.* One entity can present many identities [13];
 56 a signed session gives continuity, not scarcity. Our block tests are Wald’s
 57 sequential tests [14] run as Page’s cumulative sums [15]. Quickest change de-
 58 tection minimises delay under false-alarm constraints [16], and a CUSUM’s
 59 parameters follow from the error rates it must meet [17]; windowed statistics
 60 and confidence sequences [18] were not evaluated, so ‘testing’ below means
 61 our conversion and speed tests only. Step 7 resembles risk-based authentica-
 62 tion [19]; attacks on the code once sent are a separate problem [20, 21].

63 **3. The incident and the adapted attacker**

64 This section is motivation, not evidence: no logs from the period are in
 65 our hands, permission to describe it is pending, and no result depends on it.
 66 The flow sent an SMS to any unregistered number. The first controls, v1,
 67 were reCAPTCHA v3 on web requests, one message per number per minute,
 68 five requests per address per minute, a country allow-list, a body-length limit,
 69 a proxy block and per-platform caps. The returning abuse used thousands

70 of residential addresses, carried the iOS header to skip the CAPTCHA, and
 71 stayed under every limit.

Table 1: The v1 controls and their gaps, recalled by the operator from the incident (Recalled; not supported by logs) or found in the design review that followed (Review).

Gap	Source	v1 assumed	How it fails
A	Recalled	A platform header says which app sent the request	Sending the header reached every exemption
B	Recalled	Five requests per address per minute bound an attacker	A fresh residential address per request
C	Recalled	A number given is worth sending to	Generated numbers cost as much as real ones
D	Review	An allowed country is a safe destination	Premium ranges inside it cost far more
E	Recalled	Per-key caps bound total spend	A wide attack under every cap
F	Review	Check-then-increment is a rate limit	Two racing instances both pass
G	Review	Bulk and test flags are internal	Neither needs stronger authentication
H	Review	Each check says which check failed	Thresholds can be probed one at a time

72 *Threat model.* The attacker wants as many messages sent as possible, or
 73 sent to numbers that pay it. It buys residential proxies and human-like
 74 reCAPTCHA tokens (about 0.3 cents each), mints fingerprints, generates
 75 numbers and knows the thresholds. A pumper also has a colluding operator's
 76 ranges, where it may enter codes, report deliveries as failed, build verified
 77 history, spread over blocks and buy challenge solutions. It cannot forge an
 78 attestation, use ranges its partner does not terminate, or make a stranger
 79 enter a code.

80 4. Design

81 No message is paid for before every check passes, and cheap checks run
 82 first. Trust comes from evidence the client cannot forge, an attestation, a
 83 server-signed session and the fate of earlier codes, though such evidence can
 84 be manufactured (Section 7.8).

85 *4.1. The pipeline and the risk score*

86 A request passes the steps of Table 2 until one refuses it. Step 0 takes the
 87 platform only from a valid App Attest or Play Integrity result. Each limit and
 88 reservation is one atomic Redis operation, but the enclosing steps are not one
 89 transaction: on the normal path a refused request gives back its per-number
 90 claims and counter slot, and the hourly budget is reserved only for a message
 91 about to be enqueued, a reservation rather than a completed dispatch; a
 92 crash between them can leak reservations (Section 6). Address limits and
 93 source caps count requests, except that a request Step 11 challenges gives
 94 its source-cap count back. Every outcome after Step 0 except the challenge
 95 returns one HTTP 200 body after at least 400 ms, a floor that cannot hide
 96 longer work.

Table 2: The twelve steps (the v1 gaps of Table 1 they address in brackets). FB is the feedback loop.

Step	Check
0	Proxy block; platform from attestation, never from the header [A]
1	Signed session bound to a fingerprint; 3 requests per session per 10 min [B]
2	Limits per address, /24 and ASN, lower for hosting ASNs [B, F]
3–4	reCAPTCHA v3 threshold and origin check, web only
5	Number intelligence: cost class, sequential numbers, liveness; reads the block’s verdict stage [C, D, G]
6	Body length, excluded numbers; test bypass needs a credential [G]
7	Risk score and tier: allow, delay, challenge, downgrade, block [H]
8	Per-number backoff, 60 s doubling to 1 h, five a day
9	Per-source, platform and country caps with an adaptive multiplier
10	Operating mode: elevated above 80% of the hourly budget, emergency at 100% [E]
11	Reserve hourly count, spend and any destination slot; write state; enqueue; uniform response [E, H]
FB	Outcomes per key; destination-block tests; denylist of fine keys

97 Step 7 computes

$$s = \sum_i w_i x_i + w_c \frac{\max(0, \theta - r_{\min})}{\theta} + w_s f + w_m i - w_v v_n - w_f v_f, \quad (1)$$

98 where the x_i are request signals (CAPTCHA score, hosting ASN, fingerprint
 99 age, session requests, country mismatch, number class), $r_{\min}$ is the lowest con-
 100 version ratio over the request’s keys (address to destination block), $\theta = 0.3$,

101 $w_c = 25$, f and i flag a flooded key and one whose codes are entered within
 102 seconds, and v_n , v_f mark a number and fingerprint with verified history;
 103 other weights are 5 to 25 [22]. Tiers start at 20, 40, 60 and 80: *allow*; *delay*
 104 (a doubling wait); *challenge* (credits -20 when solved); *downgrade* (push or
 105 WhatsApp, never SMS); *block*.

106 4.2. The feedback loop

107 A send resolves as *verified* when its code is entered, *failed* after five wrong
 108 attempts or two minutes past the delivery receipt, and *undelivered* without a
 109 receipt in sixty seconds or with a failure receipt; only the first two feed rep-
 110 utation. A positive receipt after a negative one reopens the send; a late ver-
 111 ification reclassifies it, but an issued verdict stands. Each transition records
 112 its effects and its time in the same compare-and-set, and a recovery sweep
 113 finishes a dead process’s work; each effect type applies at most once, and
 114 exactly once if the sweep runs within 20 minutes. Reputation counts land
 115 in the transition’s hour and a block keeps its latest outage failure time, but
 116 a block’s statistic accumulates in processing order and a verdict is dated at
 117 the event that completes the crossing, so a late event can backdate a verdict
 118 that contains later evidence, moving its expiry and the next test’s start (by
 119 ten minutes in one diagnostic). Events arrive in time order on the live path
 120 and in every simulated run.

121 4.3. Destination policies

122 A pumper is confined to its partner’s ranges but can rotate numbers
 123 and blocks within them. Destination policies are keyed on the 8-digit block
 124 (10 000 numbers), so their protection weakens as a pumper spreads.

125 *Sequential tests.* The conversion statistic assumes users verify with $p_\ell = 0.8$
 126 and a flooder with $p_a = 0.1$: a failure adds $\ln \frac{1-p_a}{1-p_\ell} = 1.50$, a verification
 127 $\ln \frac{p_a}{p_\ell} = -2.08$. The speed statistic assumes a machine enters a verified code
 128 within five seconds with $q_a = 0.9$ and a person with $q_\ell = 0.2$, since autofill
 129 removes the keystroke [23, 24]; it adds 1.50 or -2.08 . A verdict is issued,
 130 and the statistic reset, when either exceeds $h = \ln 1000 = 6.91$. The floor
 131 $S_t = \max(-ch, S_{t-1} + \text{increment})$ sets the memory: unbounded c is Wald’s
 132 test, $c = 0$ Page’s cumulative sum; the default is $c = 1$. A first verdict makes
 133 the block’s first-time clients pass a challenge for an hour; a second within the
 134 hour moves them off SMS, refusing those without another channel.

135 *Counters.* A counter limits SMS sends per block in a daily or short refilling
 136 window. Step 11 reserves a slot and decides the stage in one atomic operation
 137 when an SMS is about to go out, so concurrent requests cannot skip the
 138 challenge. Past the limit a graded counter challenges first-time clients, and
 139 past twice the limit moves them off SMS; a refusing counter refuses. Clients
 140 with verified history are exempt from both policies, so a code entered on a
 141 pumper’s number can buy that number an exemption. A token bucket of
 142 the same rate (burst 4 or 12; its challenge-tier bucket refills independently,
 143 whereas RFC 2697’s excess bucket fills only from the committed one [25])
 144 and the counter combined with the tests were explored as alternatives.

145 4.4. Capacity controls

146 Step 9 multiplies its caps by 0.25 to 1.5 according to a baseline job com-
 147 paring each key with its expected rate, learned from the pipeline’s hourly
 148 counters; without history the static cap stands. Verified history avoids the
 149 adaptive reduction, not the static caps or the budget, which bounds SMS
 150 count and spend, not vendor or fallback costs.

151 5. Leakage estimates and a heuristic crossover

152 We compare the SMS sent to a pumper’s numbers over an attack of
 153 $T \leq 60$ minutes, by a pumper that solves no challenge and holds no verified
 154 history. Against the sequential tests, one reaching B blocks with N requests
 155 at rate λ leaks about

$$L \approx \min(N, kB + \lambda\tau), \quad k = \lceil h/1.50 \rceil = 5, \quad (2)$$

156 before its first verdict, where τ , the time a send stays unresolved, is the
 157 resolution timeout plus the delivery delay plus half the timeout worker’s
 158 period (2.55 minutes) if the carrier never verifies, and seconds if it verifies at
 159 once. Judged, such a pumper is held for the verdict’s hour, so for $T \leq 60$
 160 Equation (2) also estimates the attack’s total; it ignores prior credit and
 161 outage suspension. A carrier verifying a share σ of codes with human-like
 162 delay gives the conversion statistic an expected increment of $\sigma(-2.08) + (1 -$
 163 $\sigma)(1.50)$, zero at $\sigma^* = 0.42$; above it the statistic drifts away from a verdict.

164 A counter of q sends per block opens a window with a block’s first reserved
 165 SMS and closes it after the window length. A pumper that finds its blocks’
 166 counters empty (a cold start) sends at most

$$L_c \leq \min(N, qBw), \quad w = \lceil T/\text{window} \rceil, \quad (3)$$

167 whatever its carrier does with the codes. If a legitimate send has already
 168 opened a window on block b , using $c_b \geq 1$ slots, the pumper can also take
 169 the remaining $(q - c_b)^+$ before at most w further windows open, so in general

$$L_c \leq \min\left(N, qBw + \sum_{b \text{ open}} (q - c_b)^+\right) \leq \min\left(N, B(qw + q - 1)\right). \quad (4)$$

170 Solving challenges doubles these bounds, and verified history exempts a client
 171 from them. While neither policy’s estimate reaches N , the cold-start bound
 172 lies below the tests’ estimate when

$$B(qw - k) < \lambda\tau, \quad (5)$$

173 and a warm start adds the residual quota to the left side. Equation (5)
 174 is the algebraic crossing of two simplified quantities, neither an empirically
 175 established switching point nor a necessary condition: other limits can hold
 176 either policy below its estimate. A carrier that games outcomes may never
 177 be judged, so the tests’ leakage approaches N .

178 In the code every send takes a slot, verified clients’ included. If a block’s
 179 sends arrive as a Poisson stream with mean m per window and all reach the
 180 counter, a window holds $1 + X$ of them, $X \sim \text{Poisson}(m)$, and the idealized
 181 share beyond the quota is $E[(1 + X - q)^+]/(1 + m)$: 7.3% at $m = q/2$ and
 182 27% at $m = q$ for $q = 4$. It is not a challenge rate: exempt clients take
 183 slots without challenge, and challenge failures, the second tier and earlier
 184 gates are ignored. Equations (3)–(5) and k follow from the rules, τ from the
 185 configuration and λ from the attack; what an excess send costs a user needs
 186 the simulation.

187 6. Implementation and evaluation method

188 *Implementation.* About 4000 lines of Python implement the pipeline, feed-
 189 back loop and vendor adapters on in-memory and Redis backends. Up to re-
 190 lease 2.8.3 the simulator’s in-memory store let a sorted set expire a fixed time
 191 after its first write, whereas Redis keeps it while it is written to, so in simu-
 192 lation the outage detector and Step 5 lost recent observations. Release 2.9.0
 193 makes the in-memory store keep state as Redis does, which differential tests
 194 check operation by operation and on whole simulations, and every study was
 195 rerun with it. Earlier releases repaired seven replay and concurrency defects,
 196 each with a regression test. Of 453 tests, 19 use a real `redis-server`, eigh-
 197 teen of them testing specific interleavings of two instances, not all. Step 11 is

198 still not one transaction: a crash after its reservations leaks reserved slots or
 199 resolves the send as undelivered. The implementation is a research prototype.

200 *Simulation.* The pipeline runs against scripted vendor fakes on a simulated
 201 clock. After a warm-up of ten minutes (30 or 130 for pumping), attacks
 202 run over legitimate traffic at 20 requests per minute. Simulated users verify
 203 80% of codes after a lognormal delay (median 22 s; 2 s for the 20% using
 204 autofill); 70% of numbers are reachable on WhatsApp; 90% of users complete
 205 a challenge; 20% are returning account holders with a stable fingerprint, a
 206 trusted number and verified history, so results for returning users describe
 207 trusted identities only. Separate random streams give all scenarios at a seed
 208 the same users.

209 Three behavioural parameters have published counterparts: genuine con-
 210 version (one provider’s 68% or more [26]; ours 60 to 90%) and the never-
 211 converting destinations and identity churn of Section 2, which our non-
 212 verifying pumpers and one-use farm share; the rest are assumptions. At-
 213 tackers (Table 4) draw pool size, rate and CAPTCHA class per seed; in
 214 a nested design (ten configurations, three seeds each) these draws account
 215 for 92% [80, 98] of the variance of the farm’s leakage. Brackets are 95%
 216 percentile-bootstrap intervals, paired per seed for differences; they describe
 217 variation under this generator, not uncertainty about real traffic.

218 *Two protocols and their chronology.* Table 3 maps the studies. The first
 219 protocol, committed before its runs, fixed the matched comparison and a
 220 robustness study over twelve Latin-hypercube points (conversion 0.6–0.9 and
 221 five other behavioural parameters) and a held-out family (60 to 120 attack
 222 requests a minute, 50 000 to 500 000 addresses, heterogeneous legitimate traf-
 223 fic), with claims C1–C5 that hold if true in 90% of cells, a benchmark con-
 224 vention, not a confidence level. The matched comparison changes only the
 225 destination policy and, at three block densities, selects on tuning seeds the
 226 least leakage summed over four concentrated pumpers among settings at or
 227 above the *tuning target*, the default’s completion minus 0.5 points (the full
 228 sequential grid only at the busiest). The robustness study’s first run failed
 229 C1 in 32 of 108 cells because attack and no-attack runs drew different users;
 230 we fixed the pairing, a fix that followed a result, and reran every study. The
 231 second protocol (claims K1–K5) was written after the matched comparison
 232 had selected three counters and committed before its runs; six of its seed
 233 values had served other robustness points, never selection. Both were rerun

Table 3: The studies, their seeds (three per shifted point), attack durations and results. MC: Monte Carlo.

Study	Seeds	Attack	Results
Designs; ablation, interactions	0-29; 0-9	20 min	Table 5; Section 7.2
Dilution; score and cap sweeps; cadence	0-4; 0-9	60; 20 min	Sections 7.3, 7.4
Pumping and spread	0-9	20 (60) min	Table 6; Section 7.5
False alarms; poisoner; adaptive attackers	MC; 0-4, 0-7, 0-9	24 h; 20-70 min	Sections 7.7, 7.8
Protocol 1: matched comparison	tuning 100-102; 0-9	20 min; 24 h benign	Table 7; Figures 1, 2
Protocol 1: robustness, C1-C5	12 points	20 min; 6 h benign	Sections 7.1-7.7
Protocol 2: E1 attacked service	300-309	60 min	Table 9
E2 held-out family, K1-K5	12 points	20 min	Table 8
E3 adaptive; E4 boundary	300-309; 300-304	20-360 min	Section 7.6; Figure 3
E5 selection, every density	tuning 100-102; 300-309	20 min; 24 h benign	Section 7.6

unchanged with 2.9.0 (9 919, 7 925 and 240 runs, each carrying its code hash); against 2.8.0, total leakage changed in 855 of 17 964 comparable runs, but no claim, eligibility or selection used here did (change report in the artifact).

Designs and metrics. Seven designs: **v1** (header-trusted platform, static caps), **v1-corrected** (attested platform), **budget-only** (sessions, static caps, budget), **block-limit** (a refusing counter of five sends per block per day), **conversion-only** and **speed-only** (v2 minus one block test), and **v2**. A seed is *contained* when leakage stays at most 5% of the attack rate from some minute to the window’s end, for at least five minutes. *Attributable* loss compares the same requests with the policy enforced and absent, net of the reverse, in points of offered users: on attack-free traffic it is the *benign cost*, on the attacked trace the *extra harm* relative to the attacked no-policy arm. *Degradation* compares the attacked trace with attack-free operation under the same policy.

Table 5: Leaked share (% , mean of per-run shares) of about 600 attack requests in twenty minutes, thirty seeds, under v1, the block-limit design (BL) and v2 with source caps lifted, and under v2 with caps on, with the first-time users refused and all users completing (%). All designs, with intervals: supplementary Table S1 ([results/design_table.md](#)).

Attacker	caps lifted: leaked			caps on, v2		
	v1	BL	v2	leaked	refused	completed
Single client	5.6	0.3	0.3	0.3	1.1	79.1
Datacenter rotation	69.0	69.0	0.0	0.0	1.1	79.1
Premium pumping	99.4	1.0	0.0	0.0	1.1	79.1
Sequential numbers	99.4	1.0	11.5	10.7	3.5	77.5
Reused profile	99.5	99.5	22.5	17.4	8.6	74.2
Residential bot	37.2	37.2	37.2	29.9	16.2	69.6
Farmed CAPTCHA	99.5	99.5	99.5	45.6	54.0	45.5
Aged fingerprints	99.5	99.5	99.5	45.6	54.0	45.5
Forged header	100.0	99.5	99.5	45.6	54.0	45.5

Table 4: The 25 attacker profiles by capability, and the destination policies each faced: S, sequential tests; C, counters; B, block-limit. Names and parameters: [results/attacker_profiles.md](#).

Capability	Profiles	Policies
A signature in the request (5)	One address; datacenter rotation abroad; a premium-rate prefix; numbers walked upward; the farm with an unattested iOS header	S, B
Looks like a person (7)	Residential bot; farmed CAPTCHA with fresh or aged fingerprints; one 48-hour-old browser profile	S, B
	Low and slow, 2 or 10 requests/min over 20 ASNs; datacenter rotation buying challenge solutions	S
Games outcomes or trust (8)	Pumpers on elevated-cost or standard ranges verifying within 1 s, or 60% after 30 s	S
	Threshold-aware and receipt-faking carriers; three trust builders (500 identity/number pairs verify for 10 or 30 min, then flood)	S, C
Concentrated on the destination (5)	Four pumpers on three blocks (carrier never verifies, verifies within 1 s or 60% after 30 s, or buys challenges); a poisoner on users' blocks	S, C
Spread (generated)	3 to 300 blocks, at full rate or at most the counter's quota per block	S, C

7. Results

7.1. RQ1: what is contained, and at what cost

Datacenter rotation leaks nothing under v2 because it scores in the challenge tier and declines challenges; Step 5 refuses the premium range, and the session cap holds the single client to two messages (Table 5); in all 108 robustness cells these attacks leaked under 5% at completion within two points

254 of the paired no-attack run (claim C1). The walk leaks 61 [53, 70] messages
255 before its block’s verdict (block-limit: five), the reused profile 89 [83, 95].

256 The farm leaks 99.5% with caps lifted under every design; v2’s adaptive
257 cap holds it to 229 [215, 244] messages, 46%, by refusing 54% [47, 60] of first-
258 time users, so 45.5% complete instead of 76.4% under v1: rationing (claim
259 C5 held in 36 of 36 cells).

260 7.2. *RQ2: conditional effects of each layer*

261 In the ablation grid (90 effects, each conditional on all other layers, un-
262 adjusted intervals; artifact), removing the adaptive caps lets the farm leak
263 310 [194, 421] more messages; the risk engine, feedback loop and block key
264 hold the walk by 166, 152 and 121 messages, and the loop holds the reused
265 profile by 138. The key acts only through the loop (interaction -121 [-141 ,
266 -101]).

267 7.3. *RQ3: where reputation stops separating*

268 A farmed-CAPTCHA attacker uses an address and fingerprint once and
269 shares its country, prefix and residential ASN with every legitimate user,
270 so the penalty its failures earn lands on everyone: over an hour with caps
271 lifted (five seeds), up to twice the legitimate rate 99 to 100% of it leaks:
272 a shared key stays above 0.3 conversion until failures outnumber legitimate
273 sends about 1.7 to one, and the conversion penalty alone reaches only *delay*.
274 At ten times, 70% leaks while 17% of legitimate users are challenged and
275 3.2% refused (claim C2, at least 90% leaked with caps lifted, held in 34 of 36
276 cells). Changing the risk score one setting at a time separates no better: a
277 fresh-fingerprint weight of 40 stops the attacker only by challenging 47% of
278 legitimate users and refusing 8.5%. In the settings run, reputation separated
279 an attacker only on keys its sends dominated [13].

280 7.4. *RQ4: what rationing costs, and what it depends on*

281 When separation fails, the cap is the lever: over ten seeds the default
282 lets 47% of the farm’s requests through and refuses 41% of all legitimate
283 users, the tightest setting 20% and 64%. Both assume an oracle job told the
284 legitimate rate, run every minute; every five minutes or hourly, it lets 75 to
285 96% through. Learning the rate from the pipeline’s counters, the deployed
286 job leaks 193 [161, 229] messages with a clean three-week profile, 340 [239,
287 441] more on a cold start, 106 [62, 151] more with a stale profile and 45 [10,
288 77] more with a poisoned one.

289 7.5. Pumping against the sequential tests

Table 6: Concentrated pumpers on three blocks, caps lifted, ten seeds: messages leaked of about 574 requests in twenty minutes; seeds contained; mean minutes to containment (contained seeds) and to the first verdict (seeds with one); codes entered.

Carrier, design	Leaked (msgs)	Contained (seeds)	Containment (min)	First verdict (min)	Codes entered
never verifies: v1	570 [419, 736]	0/10	–	–	0
v2 default	90 [71, 114]	10/10	3.5 [3.1, 3.9]	3.0	0
verifies within 1 s: v2 default	19 [17, 21]	10/10	1.4 [1.1, 1.7]	0.4	19
verifies 60% after 30 s: v2 default	452 [365, 539]	0/10	–	11.6	272
v2 default, 60 min	1 067 [839, 1 310]	2/10	47.0 [42.0, 52.0]	25.5	–
buys challenges: v2 default	106 [87, 127]	10/10	4.3 [3.8, 4.9]	3.0	0

290 In Table 6 the speed test catches a carrier verifying within a second at
 291 19 messages, while the carrier verifying 60% with human-like delay is never
 292 contained: its first verdict comes at minute 11.6 on average and 452 messages
 293 leak (1 067 in an hour). Claim C3 (containment with at most 30% leaked)
 294 held in 36 of 36 cells.

295 Across 18 spread configurations leakage follows the blocks touched: with
 296 30 blocks five to seven runs in ten are still contained. In the ten unsaturated
 297 configurations Equation (2) errs by 1.6 to 7.9 messages per run (1 to 13%);
 298 saturated, it overstates by up to 91.

299 7.6. Destination policies: where the counter wins, and where it fails

Table 7: Matched comparison, first protocol, 200 blocks (144 legitimate sends per block a day), tuning seeds 100–102, evaluation seeds 0–9: twenty-minute leakage (messages) by never-verifying (NV), instant (IV), human-like (HL) and challenge-buying (CB) carriers; from 24-hour attack-free runs at 65% conversion, completion, benign cost (points), users challenged and verdict events a day. Tuning target: 63.95%.

Setting (selected as)	NV	IV	HL	CB	Compl.	Benign	Chall.	Events
none	319	566	570	405	64.13%	–	1.53%	0
sequential T1000, c1 (default)	90	19	452	106	64.12%	0.01	1.64%	6.5
sequential T300, c0 (selected at tuning target)	88	14	126	98	63.77%	0.37	6.54%	334.3
counter, graded, 4 per 10 min (selected)	24	24	24	48	64.07%	0.06	2.38%	0
counter, graded, 160 per day (selected daily)	319	409	408	405	64.11%	0.02	1.87%	0

300 *At a benign service target.* Under the first protocol (Table 7, Figures 1 and 2),
 301 the sequential setting selected at the tuning target holds the human-like
 302 carrier to 126 messages only by raising 334 false verdict events a day, and

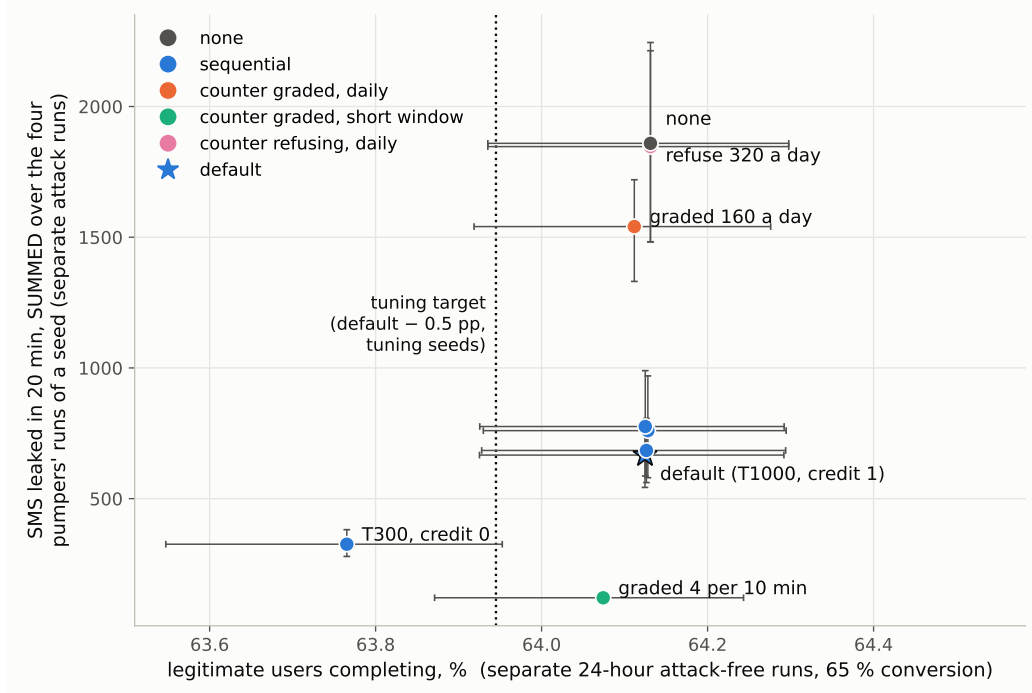


Figure 1: Matched comparison, first protocol, 200 blocks, evaluation seeds 0–9: each setting’s completion in 24-hour attack-free runs at 65% conversion against its 20-minute leakage summed over the four concentrated pumpers’ runs of a seed (not one combined attack). Bars: 95% bootstrap intervals over seeds. Dotted line: the tuning target. Unlabelled points: other sequential settings.

falls 0.18 points below the target on seeds 0–9. The graded counter of four sends per block per ten minutes leaks 24 against each outcome-reading carrier and 48 against the challenge buyer, at 0.06 points; counters selected at the sparser settings leak 6 to 18 at 0.15 to 0.35 points (up to 0.43 at 80%). With the full grid and density-specific attacks (E5) the rule selects the same counters, and so do other objectives on the tuning runs (worst-case leakage, a first-time-loss constraint, each attacker alone, 2 000 random mixes), except the instant verifier alone and 3 mixes at 200 blocks; on fresh seeds the counter leaked least of the winners against every attacker but the instant verifier (25, against 15 and 16). On seeds 300–309 the 1 000-block counter falls 0.12 points below its target, yet no seed-and-conversion cell fails the paired check (the evaluation default minus 0.5 points).

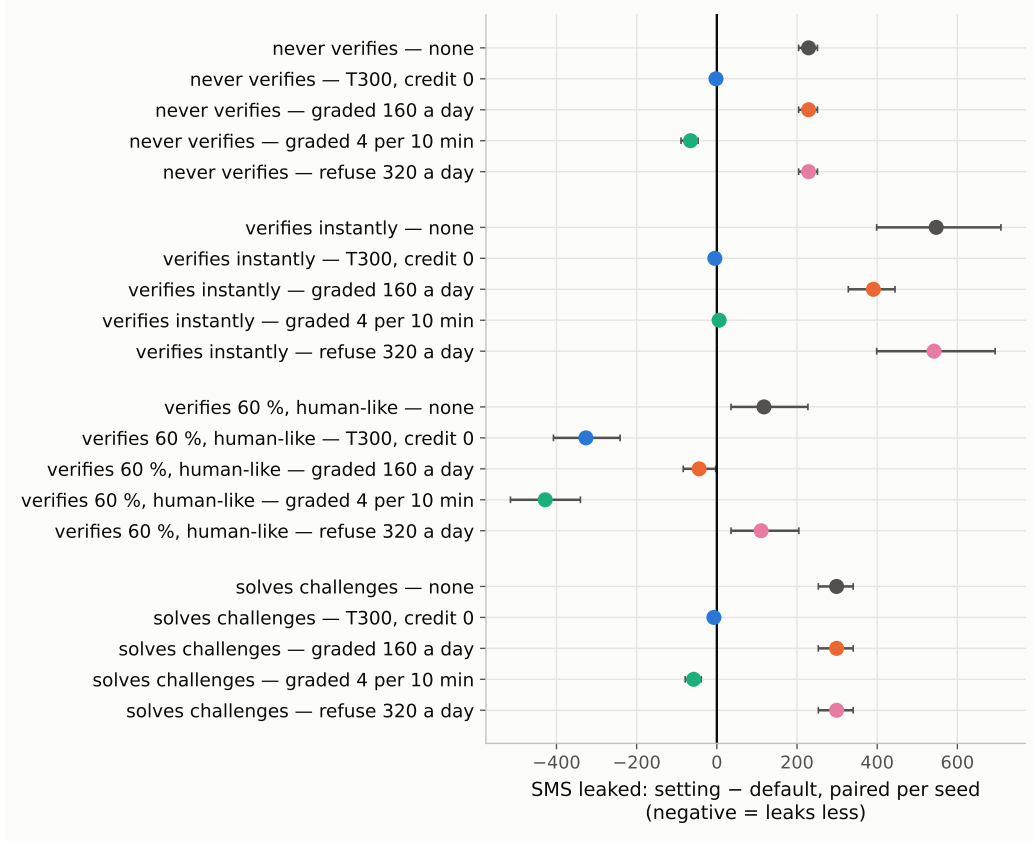


Figure 2: Matched comparison, first protocol, 200 blocks, evaluation seeds 0–9, 20-minute attacks: for each selected setting and each concentrated pumper, the setting’s leakage minus the default’s, paired per seed (negative: leaks less), with 95% bootstrap intervals over seeds.

315 *Under a second protocol.* Frozen and run on the held-out family (Table 8),
316 the counter leaked 33, 34, 33 and 62 messages against the four carriers,
317 where the default leaked 287, 36, 344 and 287 and no policy 451 to 1857, at
318 a benign cost of 0.13 [0.09, 0.17] points, and failed both service-under-attack
319 claims: 13 of the 17 cells that broke K3 lie at the three points with the most
320 legitimate traffic, where users and pumper compete for one quota; K5 failed
321 in 51 cells over 11 of 12 points. The counters selected for sparser traffic cost
322 4.7 and 5.8 points here even without an attack: the quota must match the
323 traffic.

324 Table 9 separates three questions. Against the attacked no-policy arm

Table 8: The second protocol’s claims for the principal counter (graded, 4 per 10 min) on the held-out family: 12 shifted points, held-out attack rates and pools, three seeds each, a 20-minute pumper on three blocks real users share. A claim holds if true in 90% of cells; cells share workloads and seeds and are not independent. IV: the instant verifier’s cells.

Claim	Statement	True / cells	IV	Result
K1	Leaks at most 10% of attack requests	144/144 (100%)	36/36	holds
K2	Attack-free attributable loss at most 0.5 points	36/36 (100%)	–	holds
K3	Attributable loss under attack at most 0.5 points	127/144 (88%)	32/36	fails
K4	Leaks no more than the default sequential tests	131/144 (91%)	23/36	holds pooled, not for IV
K5	Attributable loss under attack at most the default’s plus 0.25 points	93/144 (65%)	23/36	fails

the counter cost all users nothing on average and gained the attacked blocks’ users 3 points on average, since without a destination policy the pumper exhausts their blocks and the hourly budget; with the human-like carrier alone it cost them 1.9. Against attack-free operation every evaluated policy still left those users 5 to 9 points worse off (8 with none): none shields a block an attacker shares. Counter and tests together, explored outside the frozen claims, leaked least against the four carriers, but under the poisoner cost all users most of the counters and buckets and leaked more than the tuned sequential setting. A token bucket with a burst of 4 spared the poisoner’s users only by letting nearly every message through; with two untuned settings neither limiter dominates, and we rank only the implemented policies. Returning users, simulated as identities with verified history, lost nothing under these policies.

Attack-free (Figure 3a), the counter costs a block’s users nothing at one send per ten minutes, 2.3 points at four and 5.5 at eight, far below the idealized share beyond the quota (27% at four), since most challenged users complete or switch to WhatsApp; without WhatsApp eight sends cost 13.2. A 30-minute launch at three times the normal rate costs its blocks’ users 16 points. Over 60 minutes (Figure 3c) the default leaked 110, 144 and 242 messages on 3, 10 and 30 blocks, against 105, 139 and 240 from Equation (2) ($\lambda\tau \approx 90$), and the counter 71, 240 and 679 against its cold-start bound of 72, 240 and 720. No counter run that started cold (44 of 100) exceeded Equation (3); three warm ones did, by 5 or 6 messages, and none exceeded Equation (4) once 7 messages sent, exempt, to real users’ trusted numbers are set aside. With these unfitted inputs Equation (5) ordered the policies’ mean

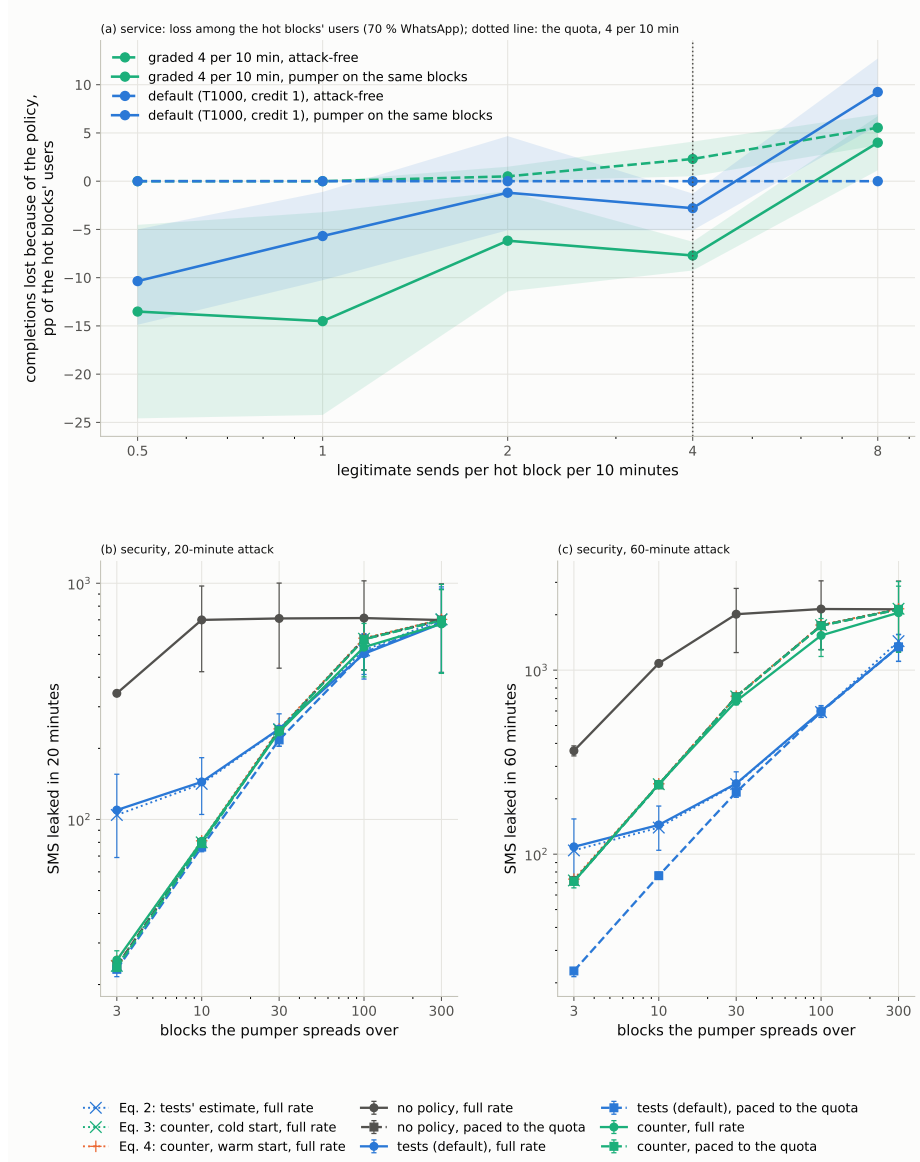


Figure 3: The counter's operating boundary (E4), seeds 300–304; lines join sampled settings and do not locate a crossover. (a) Attributable loss in points of the three hot blocks' users (70% WhatsApp reachability) against their legitimate rate, without and with a never-verifying pumper on those blocks; vertical line: the quota. (b, c) Leakage in 20 and 60 minutes against the blocks a pumper uses, at full rate or paced; dotted: Equations (2) (tests), (3) (counter, cold start) and (4) (warm start) for the full-rate pumper, from each run's inputs. Bands and bars: 95% bootstrap intervals over seeds.

Table 9: Attacked service (E1): 200 blocks, 60-minute attacks, seeds 300–309. Leaked: SMS to the attacker’s numbers, in (a) summed over the four carriers’ runs of a seed. Harm: net completions lost, in points of all users (about 1 200 a run) or of the attacked blocks’ users (block; about 18 a run), in (a) averaged over the carriers; negative: a gain. Benign: the attack-free cost against no policy. Brackets: 95% bootstrap intervals over seeds. (a) A pumper on three blocks real users share. (b) A poisoner on all 200 blocks. Per-carrier values: artifact.

(a) Four concentrated carriers							
Policy	Leaked	Extra harm vs attacked none		Degradation vs attack-free		Benign	
		all	block	all	block		
none	5 509 [4 409, 6 567]	–	–	0.19	8.0 [5.1, 10.7]	–	
sequential, default	1 149 [953, 1 340]	–0.07	–1.0 [–2.8, 0.8]	0.10	7.0 [4.7, 9.1]	0.02	
sequential <i>T</i> 300, <i>c</i> 0	397 [312, 481]	0.13	0.9 [–1.3, 3.3]	0.12	8.9 [6.0, 11.7]	0.20	
counter, 4 per 10 min	403 [386, 418]	–0.05	–3.2 [–5.0, –1.1]	0.07	4.8 [2.9, 6.7]	0.07	
counter + sequential	190 [179, 201]	–0.04	–3.1 [–5.1, –0.7]	0.07	4.9 [3.0, 6.8]	0.08	
token bucket, burst 4	424 [412, 437]	–0.12	–3.1 [–5.1, –0.7]	0.07	4.9 [2.8, 7.0]	0.00	
token bucket, burst 12	545 [531, 559]	–0.11	–2.1 [–4.3, 0.2]	0.08	5.9 [3.3, 8.8]	0.00	
(b) Block poisoner							
Policy	Leaked	Extra harm vs attacked none					
		all	first-time				
none	1 976	–	–				
sequential, default	1 357	1.09 [0.59, 1.59]	1.36				
sequential <i>T</i> 300, <i>c</i> 0	858	2.55 [2.04, 3.07]	3.20				
counter, 4 per 10 min	1 662	0.72 [0.44, 1.02]	0.91				
counter + sequential	1 286	1.43 [0.88, 1.97]	1.79				
token bucket, burst 4	1 877	0.12 [0.04, 0.23]	0.16				
token bucket, burst 12	1 974	0.00 [0.00, 0.00]	0.00				

leakage in 18 of 18 decisive cells (two ties at N), and with the warm-start term in 17, missing where the estimates differ by 0.8 messages (30 blocks, 20 minutes; measured difference 6.6 [–27.2, 42.4]); per seed, 76 of 78 decisive comparisons agree. The algebraic crossing lies near five blocks for an hour and 30 for twenty minutes, where the nearest measured differences (three and 30 blocks) do not exclude zero; it is not a measured switching point. A pumper paced to the quota leaks about as much under the counter as with no policy (71 against 72 on three blocks in an hour), and 23 under the tests. Against adaptive attackers (E3) the counter held the threshold-aware carrier, the receipt faker and the concentrated trust builder to 24 to 36 messages (502 to 665 under the default), but not trust builders on random numbers (577, as under the default); over 360 minutes the human-like carrier gained 81 messages an hour under the counter, 753 under the default.

363 7.7. *RQ5: what destination policies cost legitimate users*

364 On one block, 20 000 Monte Carlo trials per cell give the probability of
365 a verdict within 30 legitimate sends: 0.09% [0.06, 0.14] at 80% conversion
366 with 20% autofill, 1.9% at 65% and 19.5% at 50%. Over 24 hours the default
367 raises 0.5 verdict events a day on 200 busy blocks at 80% conversion and 6.5
368 at 65%, where they meet 30 of about 28 800 requests (claim C4 held in 18 of
369 18 cells).

370 A poisoner flooding for twenty minutes the twenty blocks legitimate users
371 concentrate on (caps lifted, ten seeds) raises 16 verdict events that meet 98
372 legitimate requests, 33 never completing; against the observe-only run on
373 the same trace, enforcement costs 12.7 [4.6, 22.9] completions, 3.1% of users
374 (6.3% without WhatsApp), and 23.9% with a 24-hour denylist. One that
375 stops after ten minutes leaves verdicts that meet 406.5 requests afterwards,
376 an attributable loss of 46.2 [16.9, 79.8] (48.2 over the whole run). Grading
377 limits the harm; it caps it neither at a challenge nor at the attack’s duration.

378 7.8. *Adaptive attackers, economics and overhead*

379 These scripted adaptations’ leakages are not bounds. Against v2 with
380 caps lifted, a datacenter attacker buying every challenge solution leaks 55; a
381 threshold-aware carrier and a receipt faker leak 570 each, drawing no verdict.

382 If pumpers earn a share of the 0.1422 USD retail price and pay an event-
383 level bill, they break even at a share of 0.043 under v1 and 0.054 to 0.271
384 under v2 (the instant verifier 1.310); over 360 minutes under the counter
385 the human-like carrier needs 1.048. These are break-even shares for scripted
386 attackers under assumed prices, not bounds or criminal profit.

387 On a real Redis (four vCPUs, four workers), a send costs 13.8 ms (me-
388 dian) and 21.1 ms (99th percentile). With the 400 ms floor and 50 ms ven-
389 dor calls, throughput reaches 133 requests per second at concurrency 128.
390 With fast vendors no pair of five outcome classes was distinguished (smallest
391 Kolmogorov–Smirnov $p = 0.051$; best single-threshold accuracy 53%). Under
392 an adversarial mixture with heavy-tailed vendors at concurrency 128, 81%
393 of sends overran the floor in server time, and a threshold on client response
394 times separated that run’s 2 497 sends from its 22 refusals with 88% bal-
395 anced accuracy (resubstitution); on three new runs, thresholds fitted on one
396 run scored 95 to 99.9% on the other two: a separation on that machine, not
397 a deployment accuracy.

398 8. Discussion

399 *Lessons.* In the scripted workloads, carriers that verify with human-like de-
400 lay, fake receipts or know the thresholds defeated our outcome tests but not
401 a short-window counter. Within a verdict’s hour the counter’s allowance,
402 unlike the tests’, grows with the windows an attack spans; over longer cam-
403 paigns verdicts expire and the tests leak again. Equation (5) summarizes this
404 for pumpers that solve no challenge and hold no exemption. Its inputs are
405 in an operator’s logs and timeouts; the cost of an excess send, competition
406 under attack and exemption abuse come only from the simulation. Counter
407 and tests together, with a quota set from each block’s traffic, is a hypothesis
408 for real traffic, not a recommendation.

409 *Limitations.* Everything is simulated and the calibration is assumed. Both
410 protocols were written by the author for the author’s generator, the second
411 knowing which counters had won. Some results follow from the generator (a
412 perfect premium list, a datacenter attacker that declines challenges, returning
413 users who always present verified history), and selection used an unweighted
414 sum of four scripted pumpers’ leakage and a mean service target. Close
415 rankings rest on ten or fewer seeds, and costs that round to zero are not
416 shown equivalent. Per-client quotas within a block, carrier spend ceilings,
417 bounded exemptions, tuned token buckets and other sequential statistics lie
418 outside the evaluated policies. Vendor adapters are untested live, and the
419 timing result holds only for the observer model tested. Destination policies
420 challenge people for what strangers sharing their block did, and deployment
421 under the GDPR or Saudi Arabia’s PDPL needs a lawful basis and a route
422 for refused users. Replaying real traffic is the experiment this paper most
423 needs.

424 9. Conclusion

425 In this simulation the design contains attacks that carry a signature at lit-
426 tle cost to simulated users, and rations, rather than separates, a residential at-
427 tacker that looks like a person. Counting sends on the destination leaked less
428 in aggregate than our outcome tests against concentrated pumpers, though
429 not reliably against an instantly verifying carrier; a frozen counter failed its
430 service claims when the attacker shares blocks with real users, and lost its
431 advantage against wide, paced and trust-building pumpers. Within an hour,

432 unfitted estimates order the two policies in every decisive boundary cell, as
433 an algebraic crossing, not a measured switching point.

434 **CRediT authorship contribution statement**

435 **Naveed Ahmed:** Conceptualization, Methodology, Software, Valida-
436 tion, Investigation, Writing – original draft, Writing – review & editing.

437 **Declaration of competing interest**

438 The author declares no known competing financial interests or personal
439 relationships that could have appeared to influence this work.

440 **Declaration of generative AI and AI-assisted technologies in the** 441 **writing process**

442 The author used Claude (Anthropic) for code, evaluation, drafting and
443 editing, checked numbers against the recorded results and claims against the
444 code, and takes full responsibility for the content.

445 **Data availability**

446 No human or real traffic data was used. Code, protocols (`config/`), every
447 run’s configuration and outcome (`results/`), the comparison with the 2.8.0
448 results (`store_change_report.md`) and the sources of headline numbers
449 (`headline_numbers.md`) are at
450 <https://github.com/nahmedpsu/OTP-DDOS>, release 2.9.0 (tag v2.9.0),
451 which produced the results; the 2.8.0 results are in `results/`
452 `historical_2.8`. These files are the supplementary material.

453 **References**

454 [1] MITRE Corporation, Resource hijacking: SMS pumping, technique
455 T1496.003, MITRE ATT&CK, [https://attack.mitre.org/techn](https://attack.mitre.org/techniques/T1496/003/)
456 [iques/T1496/003/](https://attack.mitre.org/techniques/T1496/003/), accessed 2026-10-01.

- [2] Commsrisk, Elon Musk says Twitter lost \$60mn a year because 390 telcos used bot accounts to pump A2P SMS, <https://commsrisk.com/elon-musk-says-twitter-lost-60mn-a-year-because-390-telcos-used-bot-accounts-to-pump-a2p-sms/>, trade press; the figure is the company's own claim (2023).
- [3] J. H. Huh, H. Shin, S. Ahn, H. Yi, J. Cho, T. Kim, M. Lim, N. Choi, Preventing artificially inflated SMS attacks through large-scale traffic inspection, in: 34th USENIX Security Symposium, USENIX Association, Seattle, WA, 2025.
- [4] M. Sahin, A. Francillon, P. Gupta, M. Ahamad, SoK: Fraud in telephony networks, in: 2017 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, 2017. doi:10.1109/EuroSP.2017.40.
- [5] M. Sahin, A. Francillon, Understanding and detecting international revenue share fraud, in: Network and Distributed System Security Symposium (NDSS), Internet Society, 2021. doi:10.14722/ndss.2021.24051.
- [6] Twilio Inc., Verify fraud guard, <https://www.twilio.com/docs/verify/preventing-toll-fraud/sms-fraud-guard>, accessed 2026-10-01.
- [7] Twilio Inc., SMS pumping risk score (Lookup v2), <https://www.twilio.com/docs/lookup/v2-api/sms-pumping-risk>, accessed 2026-10-01.
- [8] M. Motoyama, K. Levchenko, C. Kanich, D. McCoy, G. M. Voelker, S. Savage, Re: CAPTCHAs – understanding CAPTCHA-solving services in an economic context, in: 19th USENIX Security Symposium, USENIX Association, 2010.
- [9] A. Dionysiou, E. Athanasopoulos, SoK: Machine vs. machine – A systematic classification of automated machine learning-based CAPTCHA solvers, Computers & Security 97 (2020) 101947. doi:10.1016/j.cose.2020.101947.
- [10] D. Dean, A. Stubblefield, Using client puzzles to protect TLS, in: 10th USENIX Security Symposium, USENIX Association, 2001.
- [11] X. Mi, X. Feng, X. Liao, B. Liu, X. Wang, F. Qian, Z. Li, S. Alrizah, L. Chen, Y. Qian, Resident evil: Understanding residential IP proxy

- 488 as a dark service, in: 2019 IEEE Symposium on Security and Privacy
489 (S&P), IEEE, 2019. doi:10.1109/SP.2019.00011.
- 490 [12] P. Richter, F. Wolsing, A. Chandrasekaran, R. Padmanabhan, M. All-
491 man, R. Bush, A. Feldmann, A multi-perspective analysis of carrier-
492 grade NAT deployment, in: Proceedings of the 2016 ACM Internet
493 Measurement Conference (IMC), ACM, 2016. doi:10.1145/298744
494 3.2987474.
- 495 [13] J. R. Douceur, The Sybil attack, in: Peer-to-Peer Systems (IPTPS
496 2002), Vol. 2429 of LNCS, Springer, 2002, pp. 251–260. doi:10.100
497 7/3-540-45748-8_24.
- 498 [14] A. Wald, Sequential tests of statistical hypotheses, The Annals of Math-
499 ematical Statistics 16 (2) (1945) 117–186. doi:10.1214/aoms/1177731
500 118.
- 501 [15] E. S. Page, Continuous inspection schemes, Biometrika 41 (1/2) (1954)
502 100–115. doi:10.2307/2333009.
- 503 [16] V. V. Veeravalli, T. Banerjee, Quickest change detection,
504 arXiv:1210.5552 (book chapter) (2012).
- 505 [17] NIST/SEMATECH, CUSUM control charts, section 6.3.2.3, e-handbook
506 of statistical methods, [https://www.itl.nist.gov/div898/handbook](https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm)
507 [/pmc/section3/pmc323.htm](https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm), accessed 2026-10-02.
- 508 [18] S. R. Howard, A. Ramdas, J. McAuliffe, J. Sekhon, Time-uniform, non-
509 parametric, nonasymptotic confidence sequences, The Annals of Statis-
510 tics 49 (2) (2021) 1055–1080. doi:10.1214/20-AOS1991.
- 511 [19] S. Wiefeling, L. Lo Iacono, M. Dürmuth, Is this really you? An empirical
512 study on risk-based authentication applied in the wild, in: ICT Systems
513 Security and Privacy Protection (IFIP SEC 2019), Vol. 562 of IFIP Ad-
514 vances in Information and Communication Technology, Springer, 2019,
515 pp. 134–148. doi:10.1007/978-3-030-22312-0_10.
- 516 [20] C. Mulliner, R. Borgaonkar, P. Stewin, J.-P. Seifert, SMS-based one-
517 time passwords: Attacks and defense, in: Detection of Intrusions and
518 Malware, and Vulnerability Assessment (DIMVA 2013), Vol. 7967 of

- 519 LNCS, Springer, 2013, pp. 150–159. doi:10.1007/978-3-642-39235
520 -1_9.
- 521 [21] H. Siadati, T. Nguyen, P. Gupta, M. Jakobsson, N. Memon, Mind your
522 SMSes: Mitigating social engineering in second factor authentication,
523 Computers & Security 65 (2017) 14–28. doi:10.1016/j.cose.2016.
524 09.009.
- 525 [22] N. Ahmed, OTP Guard: a layered SMS validation, risk-scoring and
526 rate-limiting pipeline against OTP flood abuse (version 2.9.0), <https://github.com/nahmedpsu/OTP-DDOS>, tag v2.9.0, results produced by
527 2.9.0; 2.8.0 results (commit 1c277a0) in results/historical_2.8 (2026).
528
- 529 [23] Google LLC, SMS retriever API overview, <https://developers.google.com/identity/sms-retriever/overview>, accessed 2026-10-01.
530
- 531 [24] Web Incubator Community Group, WebOTP API, <https://github.com/WICG/WebOTP>, accessed 2026-10-01.
532
- 533 [25] J. Heinanen, R. Guerin, A single rate three color marker, RFC 2697,
534 Internet Engineering Task Force (1999). doi:10.17487/RFC2697.
- 535 [26] Twilio Inc., Verify API, <https://www.twilio.com/en-us/user-authentication-identity/verify>, accessed 2026-10-03.
536